

Especificação de Requisitos de Software

Postinder Mobile: Aplicativo de Aprovação de Conteúdo para Dispositivos Móveis

Entidade parceira: Carol Beckenstein, agência prestadora de serviços de marketing.

Qual a solução e como ela ajuda a entidade parceira?

Qual a solução de software para dispositivos móveis que pode auxiliar a entidade parceira?

Um aplicativo mobile de aprovação de conteúdo, onde a agência cria e envia postagens (imagem, vídeo, áudio, PDF) para o cliente aprovar, adorar ou pedir ajuste, direto pelo celular, sem depender de mensagens soltas.

Como o aplicativo pode auxiliar na solução da necessidade?

Hoje a Carol Beckenstein aprova conteúdo com os clientes de forma espalhada, por WhatsApp, e-mail e telefone. Isso mistura decisões com conversas comuns, não deixa histórico organizado e depende de a agência acompanhar manualmente quem já aprovou o quê. O aplicativo resolve isso centralizando esse processo num só lugar: o cliente recebe a postagem, decide com poucos toques, e a agência acompanha tudo num painel com fila consolidada e métricas por status.

1. Introdução

1.1 Propósito do documento

Este documento reúne os requisitos funcionais e não funcionais do Postinder Mobile, o aplicativo desenvolvido em React Native e Expo para esta atividade. A ideia central é simples. Hoje a aprovação de conteúdo entre a agência e o cliente acontece de forma espalhada, por WhatsApp, e-mail, telefone, sem um fluxo único. O app centraliza esse processo num só lugar, permitindo que o cliente aprove ou peça ajuste direto pelo celular.

Para dar consistência técnica ao projeto, adotamos a mesma convenção de identificação de requisitos (`RF-<módulo>-<número>` para funcionais e `RNF-<módulo>-<número>` para não funcionais) usada como referência num projeto de código aberto chamado Postinder, que trata do mesmo tipo de problema. Esse projeto de referência não está em produção nem em uso pela entidade parceira, serviu apenas de inspiração técnica e visual para este aplicativo, que foi construído do zero, com backend próprio.

1.2 Escopo do produto

O Postinder Mobile cobre o ciclo completo de uma postagem. A agência cria o conteúdo e anexa a mídia, envia para aprovação, o cliente decide (aprova, aprova com destaque de "adorei", ou pede ajuste com um motivo), a agência corrige quando necessário e reenvia, e o resultado fica visível num painel de métricas. O aplicativo atende dois perfis:

- **Cliente.** Aprova ou rejeita posts numa fila guiada. Pode entrar com login ou usar um link/token público, sem precisar de conta.
- **Agência (admin).** Cria e edita posts, gerencia clientes com CRUD completo, e acompanha a fila consolidada de aprovações junto com o dashboard de métricas.

Para viabilizar testes de ponta a ponta, foi construído um backend próprio em Node.js, TypeScript, Express e PostgreSQL. O app conversa com essa API de verdade, não com dados simulados.

1.3 Convenções

Os papéis citados no documento são **client** (cliente da agência) e **admin** (usuário interno da agência). O termo "Cliente", com C maiúsculo, sempre se refere ao usuário final que aprova conteúdo, seja por login (`/client-portal`) ou por link/token público (`/portal/:token`).

2. RF-AUTH: Autenticação e Sessão

ID	Requisito
RF-AUTH-01	O sistema deve permitir que um usuário administrativo se autentique com e-mail e senha, recebendo um par de tokens JWT (access e refresh) com contexto <code>admin</code> explícito.
RF-AUTH-02	O sistema deve permitir que um Cliente se autentique com e-mail e senha, recebendo tokens com contexto <code>client</code> . Esse contexto é distinto e não pode ser trocado pelo contexto administrativo.
RF-AUTH-03	O aplicativo deve renovar automaticamente o access token expirado, usando o refresh token armazenado. Isso acontece sem exigir novo login e sem que o usuário perceba, pois a requisição original é repetida de forma transparente.
RF-AUTH-04	O sistema deve encerrar a sessão mediante ação explícita do usuário, com o botão de logout disponível no cabeçalho das telas autenticadas.
RF-AUTH-05	O sistema deve redirecionar automaticamente para a tela inicial quando o refresh token também estiver inválido ou expirado.

3. RF-CLI: Gestão de Clientes (agência)

ID	Requisito
RF-CLI-01	O sistema deve permitir que um admin cadastre um novo Cliente com nome, e-mail, senha e cor de identidade visual.
RF-CLI-02	O sistema deve permitir listar todos os Clientes cadastrados, com indicação de status ativo ou inativo.
RF-CLI-03	O sistema deve permitir editar nome, e-mail, senha, cor e status de um Cliente existente.
RF-CLI-04	O sistema deve permitir a exclusão definitiva de um Cliente. O usuário precisa ser avisado de que os posts associados também serão removidos.
RF-CLI-05	Cada Cliente deve possuir um token de portal público único, gerado automaticamente no cadastro e utilizável no fluxo de aprovação sem login.

4. RF-POST: Gestão de Postagens (agência)

ID	Requisito
RF-POST-01	O sistema deve permitir a criação de uma Postagem vinculada a um Cliente, com título, descrição, canais (Instagram/Facebook, LinkedIn, TikTok, YouTube, Google Meu Negócio, WhatsApp, Site, E-mail Marketing) e anexo de mídia enviado do dispositivo.
RF-POST-02	O sistema deve permitir a edição de título, descrição, canais e anexo adicional de mídia de uma Postagem existente.
RF-POST-03	Toda Postagem criada deve iniciar com status "pendente de aprovação" e notificar o Cliente correspondente por push.
RF-POST-04	O sistema deve permitir o reenvio de uma Postagem rejeitada depois da correção. Isso a retorna ao status "pendente de aprovação", incrementa a revisão de conteúdo e notifica o Cliente de novo.
RF-POST-05	Ao editar uma Postagem rejeitada, o sistema deve exibir o motivo e as tags que o Cliente informou no pedido de ajuste.
RF-POST-06	O sistema deve permitir consultar Postagens filtradas por status, entre pendente, aprovado, rejeitado, enviado e executado.

5. RF-FILE: Anexos de Mídia

ID	Requisito
RF-FILE-01	O sistema deve permitir o upload de arquivos de imagem, vídeo, áudio, PDF e outros formatos de documento a partir do dispositivo do admin.
RF-FILE-02	O sistema deve inferir automaticamente o tipo do arquivo (imagem, vídeo, áudio, pdf ou documento) a partir do MIME type recebido.
RF-FILE-03	O aplicativo deve exibir imagens e reproduzir vídeos e áudios nativamente na fila de aprovação, com controles de reprodução.
RF-FILE-04	O aplicativo deve exibir PDFs embutidos no iOS. No Android, deve oferecer abertura no visualizador do sistema, já que o WebView nativo não renderiza esse formato.

6. RF-APR: Fluxo de Aprovação

ID	Requisito
RF-APR-01	O sistema deve exibir ao Cliente uma fila guiada, com uma Postagem pendente por vez. A fila avança automaticamente depois de cada decisão, até chegar ao estado "Tudo em dia".
RF-APR-02	O sistema deve permitir aprovar uma Postagem, com uma variação opcional de "Adorei" que registra reação positiva.
RF-APR-03	O sistema deve permitir solicitar ajuste em uma Postagem, exigindo motivo obrigatório e permitindo tags de classificação rápida.
RF-APR-04	Toda decisão de aprovação ou rejeição deve validar a revisão de conteúdo vigente, seguindo controle de concorrência otimista. Quando houver divergência, isto é, quando o post estiver desatualizado, o sistema deve avisar o Cliente e recarregar a fila.
RF-APR-05	O sistema deve expor à agência uma fila consolidada, somente leitura, com as Postagens pendentes de todos os Clientes e o nome de cada Cliente.
RF-APR-06	O sistema deve permitir ao Cliente consultar o histórico de decisões já tomadas, entre aprovadas, rejeitadas, enviadas e executadas.

7. RF-PORTAL: Portal do Cliente

ID	Requisito
RF-PORTAL-01	O sistema deve disponibilizar acesso à fila de aprovação tanto por login (e-mail e senha) quanto por link/token privado, seguindo as mesmas regras de decisão nos dois fluxos.
RF-PORTAL-02	O aplicativo deve permitir abrir o portal por token colando-o manualmente na tela inicial. Também deve suportar deep link nativo (<code>postinder://portal/:token</code>) quando executado fora do Expo Go.
RF-PORTAL-03	O fluxo por token não deve exigir sessão nem exibir histórico, mostrando apenas a fila pendente daquele Cliente.

8. RF-NOTIF: Notificações Push

ID	Requisito
RF-NOTIF-01	O sistema deve registrar o token push do dispositivo (Expo push token) logo após o login, associando-o ao usuário autenticado, seja admin ou client.
RF-NOTIF-02	O sistema deve notificar o Cliente quando uma Postagem nova ou corrigida entrar em estado "pendente de aprovação".
RF-NOTIF-03	O sistema deve notificar os administradores quando um Cliente aprovar ou solicitar ajuste em uma Postagem.

9. RF-FDB: Métricas e Acompanhamento

ID	Requisito
RF-FDB-01	O sistema deve exibir à agência um painel com a contagem total de Postagens e a contagem por status.
RF-FDB-02	Cada contador de status deve ser navegável, abrindo a lista filtrada das Postagens correspondentes.
RF-FDB-03	O sistema deve permitir navegar de uma Postagem listada diretamente para sua edição ou correção.

10. Requisitos Não Funcionais (RNF)

ID	Requisito
RNF-SEG-01	Senhas devem ser armazenadas exclusivamente como hash (bcrypt), nunca em texto plano.
RNF-SEG-02	Os contextos de autenticação, admin e client, devem ser segregados e não podem ser trocados entre si.
RNF-SEG-03	Toda rota administrativa deve exigir token de acesso válido. Rotas de portal por token dependem exclusivamente da validade do token presente na URL.
RNF-DESEMP-01	O upload de anexos deve ocorrer de forma direta, via multipart/form-data, com limite de 100 MB por arquivo.
RNF-DESEMP-02	Vídeos devem ser carregados sob demanda pelo player nativo, sem download integral antecipado.
RNF-USA-01	A interface deve manter uma identidade visual consistente em todo o aplicativo, com paleta de cores e tipografia únicas.
RNF-USA-02	O fluxo de aprovação deve ser utilizável inteiramente por toque, com poucos passos entre abrir o app e decidir sobre um post.
RNF-PORT-01	O aplicativo deve executar tanto em iOS quanto em Android, via Expo Go ou build nativo, sem mudanças de código específicas de plataforma além das já tratadas, como a visualização de PDF.
RNF-CONF-01	A URL da API backend deve ser configurável por variável de ambiente (<code>EXPO_PUBLIC_API_URL</code>), sem valores fixos no código.
RNF-DADOS-01	Toda decisão de aprovação ou rejeição deve ser validada contra a revisão de conteúdo vigente do post, para evitar decisões sobre conteúdo desatualizado.

11. Rastreabilidade por módulo do sistema

A tabela abaixo aponta, para cada grupo de requisitos, onde a implementação correspondente vive no código.

Módulo de código	Grupo de RF	Principais telas/rotas
<code>app/src/services/authService.ts</code> , <code>authStore.ts</code>	RF-AUTH	LoginScreen, AdminLoginScreen
<code>app/src/screens/admin/AdminClientFormScreen.tsx</code>	RF-CLI	Dashboard, botão "Novo cliente" e edição de cliente
<code>app/src/screens/admin/AdminCreatePostScreen.tsx</code>	RF-POST, RF-FILE	Dashboard, botão "Novo post" e edição de post

Módulo de código	Grupo de RF	Principais telas/rotas
<code>app/src/components/PostMediaViewer.tsx</code>	RF-FILE	Fila de aprovação, portal público
<code>app/src/hooks/useApprovalQueue.ts</code> , <code>components/ApprovalQueueView.tsx</code>	RF-APR	ApprovalQueueScreen, PublicPortalScreen
<code>app/src/screens/public/PublicPortalScreen.tsx</code> , <code>RoleSelectScreen.tsx</code>	RF-PORTAL	Tela inicial, campo de colar token
<code>app/src/services/pushService.ts</code>	RF-NOTIF	Registro após login
<code>app/src/screens/admin/AdminDashboardScreen.tsx</code> , <code>AdminPostsByStatusScreen.tsx</code>	RF-FDB	Dashboard, drilldown por status
<code>backend/src/modules/*</code>	Todos (API)	Ver <code>backend/README.md</code>